

altairsim — The graphical front panel

2026-08-11 · 53afc7b

```
cd examples/frontpanel && altairsim fp.toml
```

This is the base Altair — the machine you get by typing `altairsim` with no arguments — with one thing added: its **front-panel board is connected to altairsim-fp**, a separate program that draws the panel's lamps and switches in a window and mirrors this machine as it runs. Loading `fp.toml` prints a short note and lands you at the `altairsim>` monitor prompt, exactly like the base machine; there is no disk and no auto-boot. Drive it by hand — `MOUNT` a disk and `RUN FF00`, or single-step and watch the address, data and status lamps move.

altairsim-fp is a separate program

altairsim-fp is not part of altairsim and is not in this package. It lives in its own repository — an OpenGL rendering of the Altair's front panel that runs as a **TCP server** — and you build and run it yourself. It listens on **TCP port 8800** by default, and this machine is the client that dials it:

```
# fp.toml re-opens the base front-panel board and points it at the bridge
[[board]]
id      = "fp0"
connect = "socket:localhost:8800"
```

Get altairsim-fp from its repository and start it as that project documents. Then run this example. **Launch order does not matter:** the fp board keeps redialling `localhost:8800` on a backoff, so whether you start altairsim-fp before or after altairsim, the panel connects when it appears — and reconnects if you quit and relaunch the window.

Using it

Once both are running:

You do	What happens
Type at the <code>altairsim></code> prompt (this terminal)	The monitor runs the machine — <code>MOUNT</code> , <code>RUN FF00</code> , <code>STEP</code> , <code>EXAMINE</code> , ...
Watch the panel window	The lamps show the last bus cycle : the address and data buses, and the 8080 status word (MEMR, M1, STACK, INP, OUT, $\overline{W\bar{O}}$, INTA). At speed they blur — as they did in 1975.
Flip the on-screen sense switches (SA8–SA15)	A guest <code>IN 0FFH</code> reads them back — e.g. the DBL boot PROM checks switch A12 for the console's stop-bit count.

Two monitor commands tune or drop the link:

```
DISCONNECT fp0:gui      # unplug the panel and stop redialling
SET fp0 FPS=60          # cap how many panel updates per second cross the wire
CONNECT fp0:gui socket:HOST:PORT  # or point it at a different host/port
```

Kill the Bit — a game for the panel

```
cd examples/frontpanel && altairsim killbits.toml
```

`killbits.toml` is the same machine with a program in it: **Kill the Bit**, Dean McDaniel's 1975 front-panel game. There is no console and no disk — the panel *is* the game. A single lit bit sweeps across the upper eight **address** lamps; you kill it by flipping the **sense** switch beneath it at the instant it passes. Miss, and the bit splits into two. Before you start, put every switch **down**.

Unlike `fp.toml` , this one loads and runs on its own — its `startup` does the `LOAD` and `RUN 0` for you — and it pins the CPU to the real **2 MHz** with the host idle-sleep off, because the bit's sweep speed *is* the CPU speed and the timing is the game. Start `altairsim-fp` (before or after, order does not matter) and the bit begins to walk.

The program is `KILLBITS.ASM` (listing in `KILLBITS.PRN` , assembled to `KILLBITS.HEX`). `Displaying Data on Altair LEDs.pdf` is background on how these front-panel display tricks work.

More

The board, the sense-switch decode, and the exact wire protocol are documented in `docs/boards/mits-frontpanel.md`.